

0.83. Un son quand on ramasse : note

Le 0.81.1, plus une note : note(1, DO5, 10, 12) quand le A ramasse la pièce.

1. Ce qu'il faut comprendre

C'est le **0.81.1** (la pièce qui réapparaît), **plus un son** au moment où on la ramasse.

Ce qui est nouveau ici : `note(voix, hauteur, duree, volume)`. Elle joue une note sur la **voix 1** (la console en a deux qui chantent, 1 et 2).

• **DO5** : la hauteur, un do aigu (5 : la 5e octave). • **10** : la durée, en images, un sixième de seconde. • **12** : le volume, de 0 (muet) à 15 (le plus fort).

Elle ne bloque pas : la note joue toute seule pendant que le jeu continue.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2000 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----
```

2. UNE VARIABLE — X

```
uint8_t x = 5;          // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;          // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8
```

4. UNE VARIABLE — PX

```
uint8_t px = 15;        // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
```

5. UNE VARIABLE — PY

```
uint8_t py = 8;         // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.
```

6. UNE VARIABLE — SCORE

```
uint8_t score = 0;      // combien de pièces on a ramassées : 0 au départ

// Les places des pièces, l'une après l'autre, dans deux tableaux (le 0.32) :
//
// pièce n° :      0   1   2   3   4
// PX (colonne): 15,  3, 17,  8, 12
// PY (ligne) :   8,  2, 14,  5, 11
//
// la pièce n° 0 est en (15, 8), la n° 1 en (3, 2), la n° 2 en (17, 14)...
```

7. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PX

```
const uint8_t PX[] = { 15,  3, 17,  8, 12 };
```

8. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PY

```
const uint8_t PY[] = {  8,  2, 14,  5, 11 };
```

9. UNE VARIABLE — K

```
uint8_t k = 0;           // le numéro de la pièce EN COURS : 0, 1, 2, 3, 4, puis de nouveau 0
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(px, py, ALPHABET[15]);
    texte(0, 17, "SCORE");

    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);

        if (x == px && y == py) {
            score = score + 1;
            // Le son, morceau par morceau :
            //
            //   note(1, D05, 10, 12);
            //       | |   |   |
            //       | |   |   +-- volume : 12 (de 0, muet, à 15, le plus fort)
            //       | |   +----- durée : 10 images, un sixième de seconde
            //       | +----- hauteur : D05, un do aigu (5 : la 5e octave)
            //       +----- voix : la 1 (la console en a deux qui chantent, 1 et 2)
            //
            // La note joue TOUTE SEULE : le jeu continue pendant qu'elle sonne.
            note(1, D05, 10, 12);

            k = (k + 1) % 5;           // la pièce suivante (le 0.81.1)
            px = PX[k];
            py = PY[k];
            poser(px, py, ALPHABET[15]);
        }

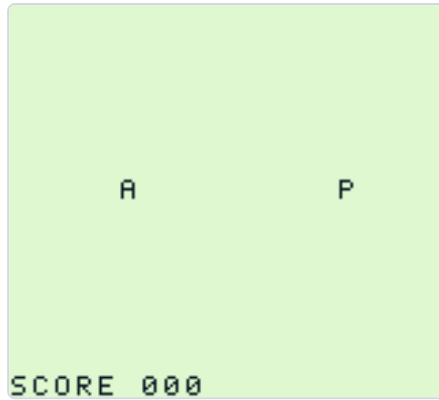
        nombre(6, 17, score);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.81.1, mais chaque pièce ramassée fait « ding ».

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la pièce ramassée
- ✓ la voix 1 a joué la note

Ensuite : **0.83.1. Un son quand on ramasse : note — et un bruit contre le mur** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++